INF3132 – Programmation Orientée Objet
S5 2023–2024

esiea

Cours 2 :

Abstraction,

Héritage,

Encapsulation

Felicia IONASCU
Siba HAIDAR
Aylen RICCA

Plan

1. Rappels
2. Abstraction
3. Allocation en mémoire
4. Héritage
5. Encapsulation
6. Cacher l'implémentation

- Allez sur wooclap.com
  - Question 1 et 2



Plan

1. Rappels
2. Abstraction
3. Allocation en mémoire
4. Héritage
5. Encapsulation
6. Cacher l'implémentation

Rappels (1/2)

- Les **classes** décrivent un set d'objets qui ont des **attributs** et des **fonctions** identiques
- La classe a une **interface**, qui établit quelles sont les requêtes qu'on peut lui faire
- **Les objets** sont des réalisations (**instances**) des classes (Ex : *Médor* est une instance de *Chien*)
- En Java, on **construit** de nouveaux objets via l'opérateur `new` et le nom de la classe et on récupère l'objet résultant dans un **référent**.

```
Chien monChien = new Chien();
```

- On accède aux attributs et aux méthodes des objets via le **référent** et l'**opérateur . (point)**

```
monChien.aboie();
```

Rappels (2/2)

- Java reprend beaucoup la syntaxe du C : les types primitifs (`int`, `double`…), les opérateurs, les embranchements, les boucles, etc.
- En Java, **tout est un objet** sauf les types primitifs `int`, `double`, `char`…
- Java introduit de nombreux objets pour les opérations courantes : `String`, `Tableaux`, `System`…
- Nous avons commencé à écrire, compiler et interpréter nos premiers programmes

Plan

1. Rappels
2. Abstraction
3. Allocation en mémoire
4. Héritage
5. Encapsulation
6. Cacher l'implémentation

Rappel : les 4 concepts de base de la POO

1. **Abstraction** – mécanisme qui permet de créer un type générique regroupant les caractéristiques communes des entités => les classes

2. **Héritage** – une nouvelle classe peut inclure les caractéristiques d'une autre classe

3. **Encapsulation** – définit les méthodes d'accès aux données et permet de cacher l'implémentation

4. **Polymorphisme** – fournir une interface unique à des entités pouvant avoir différents types

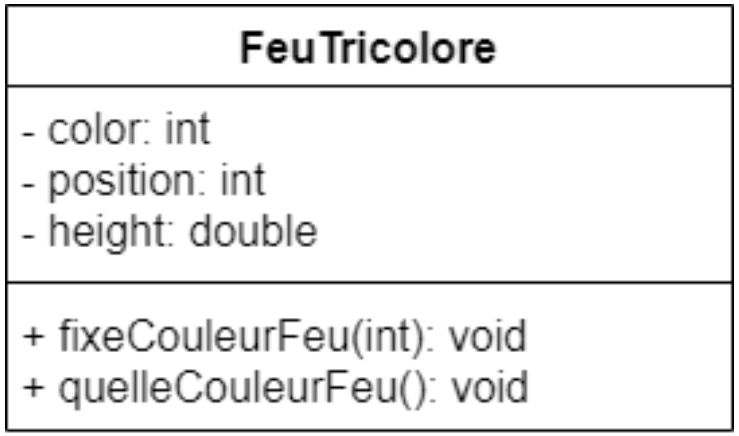
Abstraction

- Une entité peut être représentée différemment en fonction du problème qu'on doit résoudre
- Exemple : la *voiture*
  - Du point de vue du **chauffeur**, la voiture a une marque, un nombre de places, une vignette Crit'Air, un type de combustible, des commandes
  - Du point de vue d'un **constructeur de voitures**, la voiture a toutes ses pièces composantes
  - Du point de vue d'un **piéton**, la voiture a une taille, une vitesse et une distance
  - Du point de vue de l'**assureur**, la voiture a un kilométrage, un type d'usage, une marque et un domicile

- Ces **abstractions** composent le **domaine métier**
- Leur **identification** est faite par **l'analyse métier**

Structure d'une classe (1/2)

- Nouveau mot clef : `class` suivi du **nom de la classe** définit un nouveau **type** (i.e., un nouveau concept)
- La classe contient :
  - des attributs (*i.e.,* des variables)
  - des méthodes (*i.e.,* des fonctions)



Structure d'une classe (2/2)

FeuTricolore.java
(*Rappel : 1 classe, 1 fichier*)

```
class FeuTricolore {
  // Attributs
  int couleur;
  int position;
  // Méthodes
  // donne une nouvelle valeur à la couleur du feu
  void fixeCouleurFeu(int maCouleur) {
  //les attributs sont accessibles partout dans la classe, pas besoin de les
  passer en paramètre
        this.couleur = maCouleur;
        if (couleur > 2)
              couleur = 0;
  }
  //récupère la couleur actuelle du feu
  int quelleCouleurFeu() {
        return couleur;
  }
}
```

Classe, type et compilateur

- Les classes définissent des **nouveaux types** de variables :
  - pour les référents :

```
Car myCar; // Ceci est un référent
```

  - pour les objets :

```
new Car(); // Ceci est un nouvel objet.
```

- Contrat avec le **compilateur** : le compilateur peut donc vérifier l'adéquation du code
  - Mauvais type :

```
Car myCar = new Train(); // Erreur de compilation
```

  - Mauvaise méthode :

```
myCar.fly(); // Erreur de compilation
```

Constructeur (1/6)

- La classe **définit** le nombre et le type des attributs ainsi que les méthodes disponibles
- Lorsqu'on utilise l'opérateur **`new`** pour **instancier** la classe, c'est-à-dire créer un **nouvel objet** :
  - on **alloue** l'espace nécessaire en mémoire
  - on **initialise** les valeurs particulières de ses attributs

Cette initialisation est confiée au **constructeur**, une méthode particulière **de même nom que la classe** et **sans type de retour** (même pas **`void`**)

Constructeur (2/6)

- La création d'un nouvel objet se fait en combinant l'**opérateur** `new` qui effectue une **allocation dynamique** dans la mémoire des objets et l'**appel du constructeur** qui **initialise** les attributs.

```
/* Déclaration du référent */
TrafficLight firstTrafficLight;
/* Allocation et initialisation de l'objet */
firstTrafficLight = new TrafficLight(0, 3, 2.0);
```

Afin de garder un lien vers notre nouvel objet, il est d'usage de le renvoyer vers un référent déclaré préalablement

Constructeur (3/6)

- Le constructeur est appelé **automatiquement** pour chaque objet crée
- **L'initialisation** de l'objet est garantie par l'appel du constructeur
- Si la classe n'a pas de constructeur défini, le compilateur en crée un **par défaut**

- Les attributs des objets qui ne sont pas initialisés spécifiquement sont initialisés automatiquement avec la **valeur par défaut** de leur type
  - `false` pour les `boolean` ;
  - `0` pour tous les types primitifs numériques ;
  - `null` pour tous les types classe

Constructeur (4/6)

- Le constructeur n'a pas de type de retour

```
class TrafficLight {
   // Fields declaration
   int color;
   int position;
   double height;

   // Constructor (! no return type !)
   TrafficLight(int c, int p, double h) {
      color = c;
      position = p;
      height = h;
   }
}
```

Constructeur (5/6)

```
class TrafficLight {
   // Fields declaration
   int color;
   int position;
   double height;

   // Constructor (! no return type !)
   TrafficLight(int color, int position, double height) {
      this.color = color;
      this.position = position;
      this.height = height;
   }
}
```

- `this` (l'objet lui-même) permet de **distinguer** les attributs des arguments de même nom

Constructeur (6/6)

- Remarque : L'ordre des paramètres du constructeur détermine l'ordre des arguments lors de l'appel à **new**

```
// Constructor (! no return type !)
TrafficLight(int color, int position, double height) {
   /* ... */
}
```

```
TrafficLight firstTL = new TrafficLight(0, 3, 2.0);
```

Constructeur par défaut

```
class Street {
	int i;
}
class House {
	House (int val) {}
}

class DefaultConstructor {
	public static void main(String[] args) {
		Street st = new Street();	//default called
		House ho = new House();		//compiler error
	}
}
```

Surcharge de constructeur (1/3)

- Problème :
  - on veut une classe qui s'initialise soit avec une information standard, soit en lisant l'information d'un fichier
- Contrainte :
  - le constructeur doit avoir toujours le nom de la classe et pas de valeur de retour
- Solution :
  - Deux constructeurs : un par défaut et le deuxième avec un String en paramètre (le nom du fichier)
- => On **surcharge le constructeur**

Surcharge de constructeur (2/3)

- La définition de la **surcharge d'une méthode :**
  - Même nom de méthode
  - Nombre diffèrent de paramètres ou
  - Type(s) diffèrent(s) de paramètres
  - Le type de retour ne compte pas

- On peut déclarer autant de constructeurs qu'on veut

- Le langage permet l'utilisation de maximum 255 paramètres dans une méthode (donc aussi dans un constructeur)

- … mais **7 paramètres** c'est déjà beaucoup et on doit modifier la conception de la classe si on arrive à cela

Surcharge de constructeur (3/3)

```
class FileReader {
	String value;
	 String   folderName;
	int maxChar;
	FileReader() {                          //there is no value yet
		value = "";
	}
	FileReader(String fileName) {           //file name given as parameter
		value = fileName;
	}
	FileReader(String fileName, String folderName, int maxChar) {
                                                //file name given as parameter
		value = fileName;
		this.folderName = folderName;
		this.maxChar = maxChar;
	}
}
```

La destruction d'un objet

- En C++, toutes les classes ont un destructeur qui est appelé automatiquement quand un objet est détruit avec `delete`
- En C++, tous les objets construits avec `new` doivent être détruits avec `delete`

- En Java, **il n'y a pas de destructeur**

- La mémoire occupée par un objet est libérée par un mécanisme de la JVM : le Ramasse-Miettes (*Garbage Collector*)

Quiz

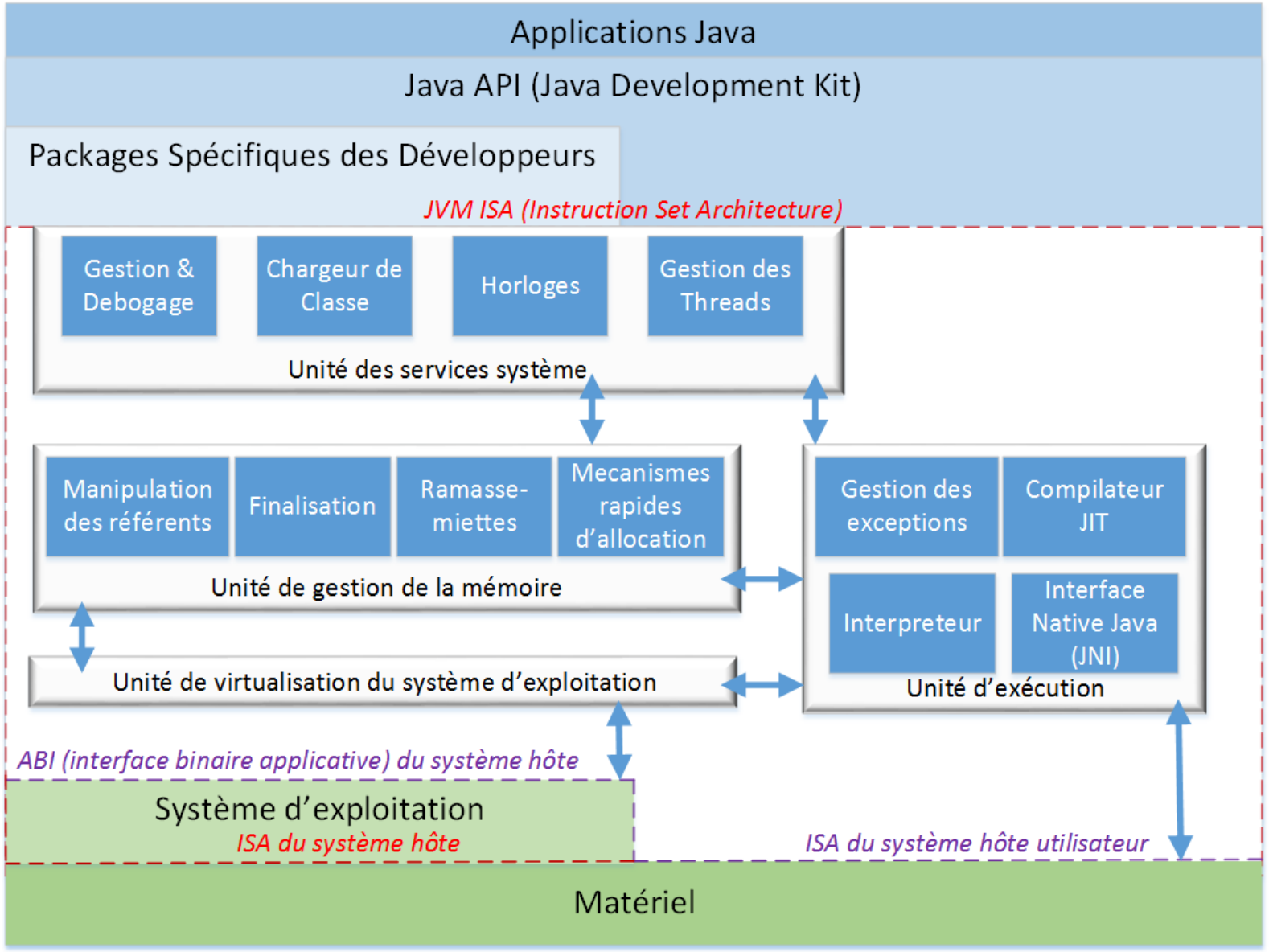


Cours 2 - Question 1
Question 2 et
Question 3

Plan

1. Rappels
2. Abstraction
3. Allocation en mémoire
4. Héritage
5. Encapsulation
6. Cacher l'implémentation

L'architecture de la JVM



Cotroneo, D., Pietrantuono, R., Mariani, L., & Pastore, F. (2007, September). Investigation of failure causes in workload-driven reliability testing. In 4th int. workshop on Software quality assurance (pp. 78-85).

Image source : https://www.researchgate.net/publication/221388386_Investigation_of_failure_causes_in_workload-driven_reliability_testing/figures?lo=1, copyright R. Pietrantuono

Les types de mémoire

- Registres
  - Dans le processeur : sont alloués par le compilateur
  - Pas de contrôle direct en Java
- La Pile (*the Stack*)
  - Dans la RAM, avec support du processeur via un pointeur (*stack pointer*)
  - Gérée par le compilateur qui doit savoir la taille et la durée de vie des données
- Le Tas (*the Heap*)
  - Dans la RAM, zone mémoire d'usage général
  - Le compilateur n'a pas besoin de savoir ni la taille, ni la durée de vie des objets
- Le stockage statique
  - Dans la RAM, dans une zone fixe
  - Les données sont stockées pour la durée de vie du programme
- Le stockage constant
  - Dans le code du programme ou dans la ROM (read-only memory)
- Le stockage non-RAM
  - Sur disque ou en réseau : objets stream ou objets sérialisés

Allocations de variable

Quoi	Où	Comment
types primitifs et référents	Pile, RAM	Allocation automatique
Objets	Tas, RAM	Allocation manuelle avec `new`
Objets statiques	Stockage statique, RAM	Déclaration avec le mot-clé `static`
Constantes	Program bytecode ou ROM	Déclaration avec le mot-clé `final`
Streams, objets sérialisés	Disque ou réseau	En utilisant les bibliothèques Java

Allocation sur la pile

- Comme en procédural, les variables déclarées dans un bloc (entre { }) sont allouées sur la **pile** (*stack*)
- Lorsque le bloc se termine, les variables sont dépilées et la mémoire est libérée
- Attention ! en Java, uniquement **les types primitifs** et **les référents** sont alloués sur la pile

```
void blink(int nbTimes) { // allocating nbTimes
   //count is allocated inside the for's block
   for(int count = 0; count < nbTimes ; count++) {
      System.out.println("Light on");
      System.out.println("Light off");
   } //disallocating count
} //disallocating nbTimes
```

Allocation sur le tas

- En procédural, on allouait manuellement de grands volumes de données (tableaux, structures) sur le tas avec `malloc`
- En POO, on alloue (et initialise) nos objets manuellement sur le tas avec `new`

- **Le tas (*heap*),** c'est ce que nous appelions la **mémoire des objets**

Les objets qui sont dans le tas ne disparaissent pas une fois le bloc terminé !

Allocation sur le tas en Java

- En Java, l'allocation d'**objets** se fait **toujours sur le tas**
- L'opérateur **new** renvoie un **référent** sur le nouvel objet

```
Car myCar = null; // Referent
myCar = new Car(); // Constructor, myCar is allocated then
     attributs are initialized by the Car constructor.
myCar.go(); // Method call
```

- En Java, il n'y a **pas de désallocation explicite** de mémoire (comme en C++)
- On considère qu'un objet **peut** disparaître lorsque plus aucun référent ne pointe sur lui

- Java implémente une routine, appelée **ramasse miettes** (*Garbage Collector*) dont le rôle est de **détecter** et de **supprimer** ces **objets non-référencés**

Ramasse-miettes - Java (1/3)

```
Car firstCar = new Car("red"); //Alloc. of red car
Car secondCar = new Car("blue"); //Alloc. of blue car
secondCar = firstCar; //No more link on blue
                      //blue can be deleted
firstCar = null; //Still a link on red
secondCar = null; //No more link on red
                  //red can be deleted
System.gc(); //Effective deletion
```

- Le ramasse-miettes évite la quasi-totalité* des fuites mémoire
  - *Le ramasse-miettes ne gère pas le code natif (C/C++) qui est appelé depuis un programme Java

Ramasse-miettes - Java (2/3)

- Le ramasse-miettes est exécuté :
  - **régulièrement**
  - quand la mémoire **manque**
  - sur appel **explicite** à `System.gc()`
- Il doit :
  - **détecter** et **libérer** les objets non-référencés
  - **détecter** les **cycles** (A pointe vers B et B vers A, mais rien vers A et B)
  - **optimiser** l'espace mémoire



Ramasse-miettes - Java (3/3)

- Le Ramasse-miettes appelle la méthode `finalize()` sur chaque objet
- `System.gc()` force la finalisation des objets
- Toute mémoire allouée par un mécanisme différent de **`new`** doit être désallouée dans la méthode `finalize()`
  - Ex. : les zones de mémoire allouées dans les méthodes natives C et C++ via la Java Native Interface (**JNI**)
- On peut utiliser la méthode `finalize()` pour détecter des bugs – la *Death Condition*
  - Ex. : des fichiers laissés ouverts alors que l'objet qui les gère est traité par le R-M

Différence pile et tas en Java

	Pile	Tas
Application	Utilisée une part à la fois pendant l'exécution d'un thread. Thread-safe (chaque thread opère dans sa pile)	Ressource partagée: toute l'appli utilise l'espace tas Non thread-safe (on doit utiliser des mécanismes de synchronisation)
Taille	Limitée par l'OS, plus petite que le Tas	Définie par l'utilisateur ou taille par défaut, pas de limite OS
Stockage	Variables primitives et référents vers des objets dans le tas	Tous les objets créés avec **new**
Ordre	LIFO (last in, first out)	Méthodes de gestion complexes (Garbage Collector)
Durée de vie	L'exécution de la méthode courante	L'exécution de l'application
Efficacité	Relativement plus rapide	Relativement plus lente
Allocation/ Désallocation	Appel de la méthode / retour de la méthode	Création des objets avec new / destruction par le Garbage Collector quand l'objet n'est plus référencé

Allocation statique (1/3)

- En Java, l'allocation statique est déclarée par le qualificatif

```
static
```

- Deux situations d'usage :
  - On a besoin d'un espace de stockage pour une donnée qui est partagée par tous les objets d'une classe et même en absence de création d'objet
  - On a besoin d'une méthode qui n'est pas associée avec un certain objet mais avec la classe

- Les éléments statiques n'ont pas d'accès à des éléments non-statiques
- Un élément d'un objet peut être statique, mais l'objet sera toujours alloué dans le tas

Allocation statique (2/3)

- Exemple : attribut statique

```
class StaticAttribute {
        static int i = 12;
}
/*dans une autre classe:*/
StaticAttribute st1 = new StaticAttribute();
StaticAttribute st2 = new StaticAttribute();
//ici, st1.i et st2.i ont tous les deux la valeur 12

st1.i++;
//ici, st1.i et st2.i ont tous les deux la valeur 13

st2.i++;
//ici, st1.i et st2.i ont tous les deux la valeur 14

StaticAttribute.i++;
//ici, st1.i et st2.i ont tous les deux la valeur 15
```

Allocation statique (3/3)

- Exemple : méthode statique

```
class StaticMethod {
        static void increment() {
                StaticAttribute.i++;
         }
}
class StaticAttribute {
        static int i = 12;
}

/*dans une autre classe*/
StaticMethod sm = new StaticMethod();
sm.increment();
//ici, i a la valeur 13

StaticMethod.increment();
//ici, i a la valeur 14
```

Exercice



Cours 2 - Question 4

- Attribut statique

Note sur les méthodes

- Le code des méthodes est stocké **dans le tas interne de la JVM**
- Le tas interne n'est pas libéré par le ramasse-miettes
- Les méthodes sont chargées au démarrage du programme et restent inchangées jusqu'à la fin du programme

En résumé (1/2)

- Le constructeur est une méthode :
  - de même nom que la classe
  - sans type de retour (même pas `void`)
  - qui initialise **TOUS** les attributs de la classe
- Pour instancier un nouvel objet on appelle `new` constructeur

```
Car c = new Car("2CV", "rouge");
```

- Java garantit que chaque classe a un constructeur par défaut, même si le programmeur n'en a pas créé un

En résumé (2/2)

```
{
     int doors = 2;
     Car c = new Car("2CV", "rouge“, doors);
     c = null;          //l’objet lui-même sera détruit par le GC
}                       //l’entier doors et le référent c sont désalloués de la pile
```

- Un objet est détruit automatiquement (par le ramasse-miettes) lorsque qu’il n’a plus de référent
- La durée de vie des éléments statiques est celle du programme
- Les types primitifs et les référents sont alloués sur la pile
- Les objets sont alloués sur le tas
- Les méthodes sont allouées dans le tas interne de la JVM

Plan

1. Rappels
2. Abstraction
3. Allocation en mémoire
4. Héritage
5. Encapsulation
6. Cacher l'implémentation

Rappel : les 4 concepts de base de la POO

1. **Abstraction** – mécanisme qui permet de créer un type générique regroupant les caractéristiques communes des entités => les classes

2. **Héritage** – une nouvelle classe peut inclure les caractéristiques d'une autre classe

3. **Encapsulation** – définit les méthodes d'accès aux données et permet de cacher l'implémentation

4. **Polymorphisme** – fournir une interface unique à des entités pouvant avoir différents types

1 concept = 1 classe

- Nous avons dit précédemment que nous allions créer **une classe pour chaque concept** (une classe Voiture, une classe Route)

Que faire si plusieurs concepts se recouvrent ?

- un Étudiant et un DéléguéDeClasse
- un Cercle, un Rectangle et un Losange

Solution naïve (1/2)

```
class Student {
      String firstName;
      String name;
      int year;

      void attendTo (Course c);
      void doTheir (Homework w);
}
```

```
class ClassRep {
      String firstName;
      String name;
      int year;
      boolean isRepresentative;

      void attendTo (Course c);
      void doTheir (Homework w);
      void informTheir(ClassMate cm);
      void participateTo (Jury j);
}
```

Solution naïve (2/2)

```
class Student {
      String firstName;
      String name;
      int year;

      void attendTo (Course c);
      void doTheir (Homework w);
}
```

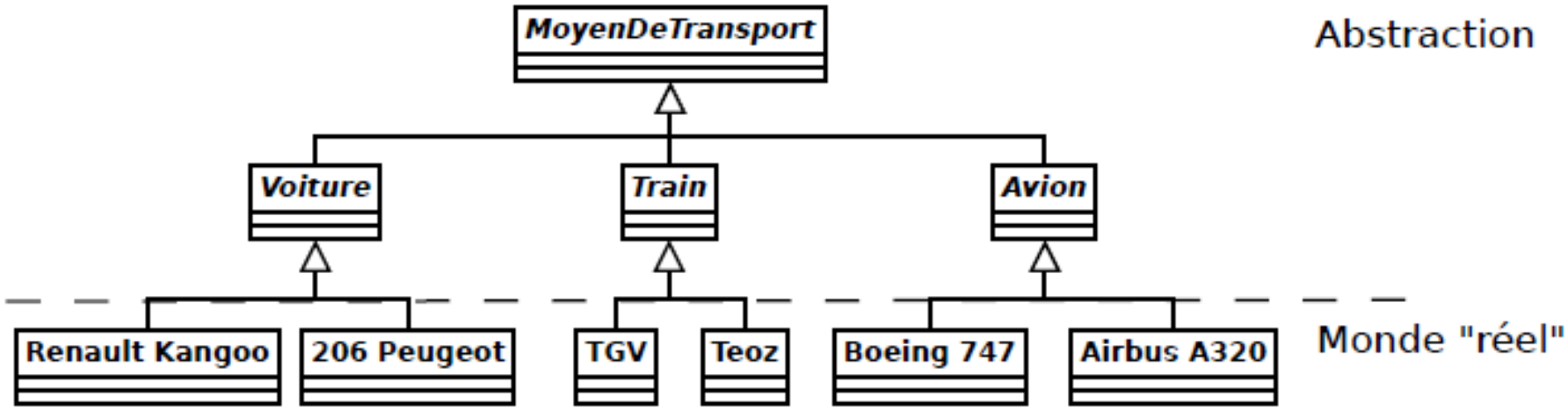
```
class ClassRep {
      String firstName;
      String name;
      int year;
      boolean isRepresentative;

      void attendTo (Course c);
      void doTheir (Homework w);
      void informTheir(ClassMate cm);
      void participateTo (Jury j);
}
```

- On observe visiblement une **redondance de code** lié au **recouvrement** des deux concepts (tous les `ClassRep` sont aussi des `Student`)

Taxonomie

- Lorsque nous parlons, nous utilisons simultanément plusieurs **échelles de pensée**
  - La voiture qui passe est à la fois : une 206 Peugeot, une voiture, un véhicule roulant, un moyen de transport, un objet du monde, . . .
- Toutes ces échelles forment entre elles une **hiérarchie** (appelée taxonomie) :
  - du plus **générique** (abstrait) au plus **spécifique** (monde réel)

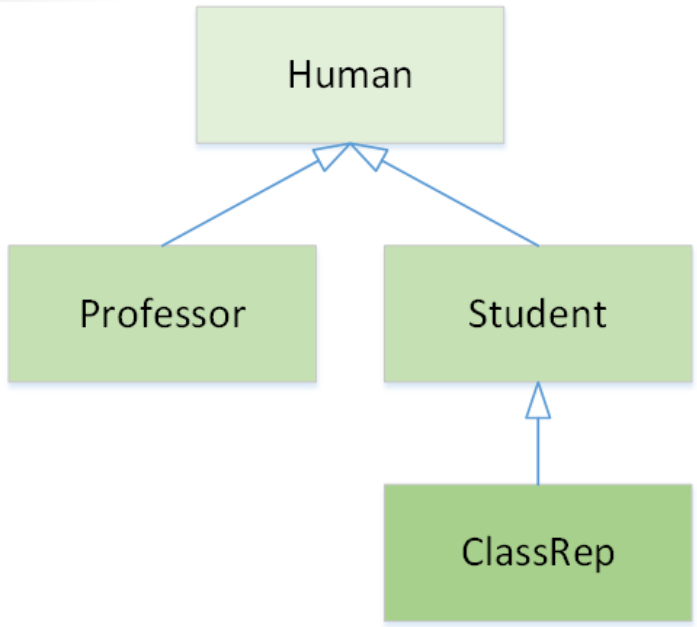


Et mes étudiants ? (1/2)

- Dans notre exemple précédent on peut dire que les *Délégués de Classe* sont des cas particuliers des *Étudiants* (**spécialisation**)

- On pourrait dire à l'inverse que tous les *Étudiants* et les *Professeurs* sont tous des *Humains* (**généralisation**)

Et mes étudiants ? (2/2)



- On en déduit que les *Délégués de Classe*, les *Étudiants*, les *Professeurs* et les *Humains* partagent des caractéristiques (attributs) et des capacités (méthodes) **communes**
- On en déduit aussi qu'ils possèdent des caractéristiques et des capacités **différentes** justifiant des concepts séparés

- Certaines classes possèdent visiblement une relation de filiation (tous les Délégués sont des Étudiants) et d'autres non (Professeurs et Étudiants sont disjoints)

Héritage (1/4)

- Lorsque l'on désire qu'une classe possède **au moins tous** les attributs et méthodes d'une autre classe, on va dire quelle **hérite** de cette classe
- Cela va nous permettre d'éviter de réécrire les attributs et les méthodes communs aux deux classes :

  c'est de la **factorisation** de code

Héritage (2/4)

```
class Student {
      String firstName;
      String name;
      int year;

      void attendTo (Course c);
      void doTheir (Homework w);
}
```

```
class ClassRep extends Student {
      boolean isRepresentative;

      void informTheir(ClassMate cm);
      void participateTo (Jury j);
}
```

- ClassRep hérite donc de ses attributs name, etc. et l'on peut écrire :

```
ClassRep rep = new ClassRep();
System.out.println("Le delegue est "+rep.forname);
```

Héritage (3/4)

- La classe qui hérite est dite **classe fille**
- La classe dont on hérite est dite **classe mère**
- La classe fille possède au moins tous les attributs et méthodes de la classe mère

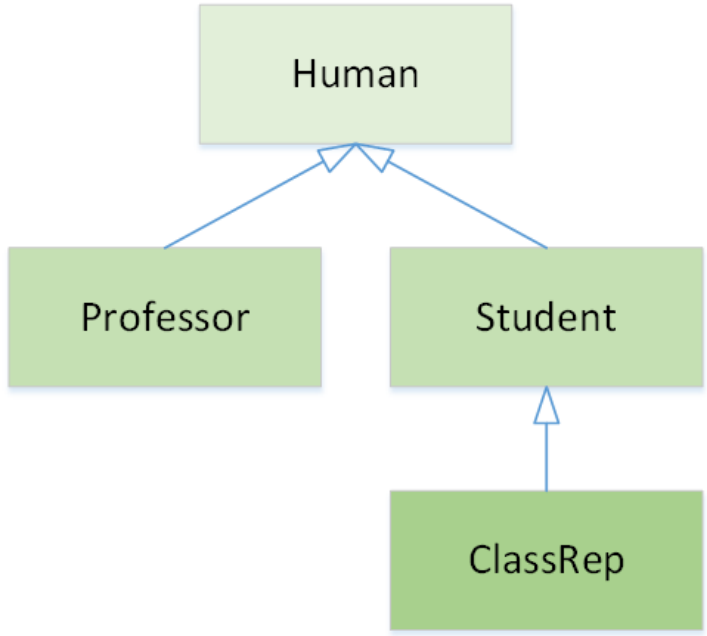


- Si l'on sait qu'un mammifère a des mamelles, et qu'un dauphin est un mammifère, alors qu'en déduit-on ?
- Le dauphin a des mamelles
- Si l'on sait qu'un dauphin a des nageoires, et qu'un dauphin est un mammifère, alors qu'en déduit-on ?
- Un dauphin est un mammifère qui a des nageoires

- => Le dauphin (classe fille) est un mammifère (classe mère) avec des caractéristiques en plus

Héritage (4/4)

- Si deux classes possèdent des attributs/méthodes communes mais que chacune a des spécificités qui lui sont propres : elles ne peuvent pas hériter l'une de l'autre (ex. : *Professeur* et *Étudiant*)
- En revanche elles peuvent hériter d'une troisième classe qui correspond à leur partie commune (ex. : *Humain*)



En résumé

- L'**héritage** est un pilier fondamental de la Programmation Orienté Objet car il pousse à définir les classes comme des empilements de concepts du plus **spécifique** au plus **générique**

- Cela permet de **factoriser le code** : pas de répétition inutile, plus lisible, plus facile à corriger, etc.

- Cela permet d'augmenter la **réutilisabilité du code**
  - La classe Peugeot206 a peu de chance d'être réutilisée mais sa classe mère Voiture peut servir dans de nombreux programmes !

Exercice



Cours 2 - Question 5

- Héritage

Héritage en Java (1/4)

- On spécifie la relation d'héritage entre les classes au moment de la définition de la classe fille :

```
class Car extends Vehicle
```

- En Java, une classe ne peut hériter que d'une **seule** classe mère

- Deux moyens pour différentier la classe fille de la classe mère :
  1. Le rajout d'attributs ou de méthodes dans la classe fille
  2. La surcharge des méthodes de la classe mère, dans la classe fille
     - => changement de comportement

Héritage en Java (2/4)

```
class Vehicle {
    int position;
     void go() {
       position = 0;
        }
}

class Plane extends Vehicle {
   void  land() {
        position = 0;
    }
}
```

Rajout de méthode

```
class Vehicle {
   int position;
   void go() {
       position = 0;
   }
}

class Plane extends Vehicle {
   void go() {
        position = 1;
    }
}
```

Redéfinition de méthode

Héritage en Java (3/4)

- Toutes les classes Java héritent, de manière directe ou indirecte, de la *super-classe* `Object`
- Toutes les classes qu'on écrit héritent aussi de la classe `Object`
- On peut utiliser les méthodes d'`Object` en les surchargeant, afin de les rendre spécifiques pour notre classe

- Deux méthodes très utiles :

```
public boolean equals(Object obj)
    //Indique si un objet est « égal à » l'objet courant
    //par défaut, on compare des références

public String toString()
    //Retourne une représentation de type String de l'objet courant
```

La classe Object

- Classe mère de toutes les classes Java (dans le JDK ou écrites par nous)

```
public class Object {
     public Object() {...}  // constructeur
     public String toString() {...} //utilisée par la JVM quand un objet doit
                                    //être représenté en tant que String
     protected native Object clone()  throws CloneNotSupportedException {...}
     public equals(java.lang.Object) {...}
     public  native  int hashCode() {...}
     protected  void finalize()  throws Throwable {...}
     public  final  native Class getClass() {...}
     // méthodes utilisées dans la gestion des threads
     public final  native  void notify() {...}
     public final  native  void notifyAll() {...}
     public final  void wait(long)  throws InterruptedException {...}
     public final  void wait(long,  int)  throws InterruptedException {...}
}
```

Héritage en Java (4/4)

```
public class Book {
	String ISBN;
	String auteur;
	String titre;
	String getISBN() return ISBN;

	//on surcharge la méthode equals de la super-classe Object pour
	//préciser sur quel critère deux livres sont égaux
	public boolean equals(Object obj) {
		if (obj  instanceof  Book)   //on vérifie le type de objet
			//on utilise la méthode equals de la classe String
			return ISBN.equals((Book)obj.getISBN());
		else
			return false;
	}
}
```

Le qualificatif `final` (1/3)

- Le qualificatif `final` agit différemment :
  - Sur un type primitif : rend la valeur constante
  - Sur un référent : rend le référent constant (pas l'objet)

- Le qualificatif `final` peut être utilisé pour :
  - Attribut : la valeur ne peut pas être modifiée; elle doit être initialisée
  - Méthode : la méthode ne peut pas être surchargée
  - Classe : **empêcher qu'une classe soit dérivée**
    - Ex. : `public final class String`

Le qualificatif `final` (2/3)

- Pourquoi voudrait-on empêcher qu'une classe soit dérivée ?
  - Le design de la classe est tel qu'il n'y a jamais besoin de la changer
  - Pour des raisons de sécurité

- Allez sur wooclap.com
  - Question 3



Le qualificatif `final` (3/3)

```
class Value {
        int i = 1;
}
public class FinalData {
        static final int VAL_TWO = 99;                  //Constante à la compilation
        static final int i2 = (int)(Math.random()*20);  //Ne peut pas être constante à la compilation
        Value v1 = new Value();
        final Value v2 = new Value();
        final int[] a = { 1, 2, 3, 4, 5, 6 };
        public static void main(String[] args) {
                FinalData fd1 = new FinalData();
                fd1.v2.i++;                             //L'objet n'est pas constant ! (Le référent est constant)
                fd1.v1 = new Value();                   //OK -- pas final
                for(int i = 0; i < fd1.a.length; i++)
                                fd1.a[i]++;             //L'objet n'est pas constant!
                //! fd1.v2 = new Value();               //Erreur : le référent ne peut pas être changé
                //! fd1.a = new int[3];                 //Erreur : le référent ne peut pas être changé
        }
}
```

Héritage : le mot-clé super

```java
public class BaseClass {
    protected String name;
    protected int id;
    public BaseClass(String name, int id) {
        this.name = name;
        this.id = id;
    }
    @Override
    public String toString() {
        return name + " " + id;
    }
}
```

```java
public class ChildClass extends BaseClass {
    public int age;
    public ChildClass(String name, int id, int age) {
        super(name, id);
        this.age = age;
    }
    @Override
    public String toString() {
        return super.toString() + " " + age;
    }
}
```

```java
public class Main {
   public static void main(String[] args) {
      ChildClass cc = new ChildClass("Julia", 13612, 12);
        System.out.println(cc);
   }
}
```

- **super** fait référence à un objet de la classe mère
- On l'utilise pour avoir accès :
  - aux **méthodes** de la classe *mère*
  - et donc au **constructeur** de la classe *mère*

Exercices

- Allez sur wooclap.com
  - Question 4 et 5



Cours 2 - Question 6

Plan

1. Rappels
2. Abstraction
3. Allocation en mémoire
4. Héritage
5. Encapsulation
6. Cacher l'implémentation

Rappel : les 4 concepts de base de la POO

1. **Abstraction** – mécanisme qui permet de créer un type générique regroupant les caractéristiques communes des entités => les classes

2. **Héritage** – une nouvelle classe peut inclure les caractéristiques d'une autre classe

3. **Encapsulation** – définit les méthodes d'accès aux données et permet de cacher l'implémentation

4. **Polymorphisme** – fournir une interface unique à des entités pouvant avoir différents types

Encapsulation (1/3)

- Le principe général de l'**encapsulation** consiste à dire que les attributs de chaque classe lui sont **privés** (`private`)
  - C'est-à-dire qu'ils ne sont pas accessibles aux autres classes

- Une méthode **privée** ne peut être appelée que par les objets de la classe elle-même
  - c'est une méthode à usage interne

- On peut déclarer certains attributs ou méthodes **publiques** (`public`) pour signifier qu'ils peuvent librement être lus / modifiés / appelés par d'autres objets

Encapsulation (2/3)

- Désormais, nous préciserons systématiquement le statut (`private` ou `public`) de chaque attribut ou méthode !

```
public class Mario {          //the class must be public to be used by another class
   private int nbLife;        // no other but the Mario class can access this
   public Mario() {           // constructor must be public to be called
      nbLife = 3;
   }

   private void checkDead() {                // private method for internal use
        if (nbLife <= 0) {
          System.out.println ("Game Over!");
        }
   }

   public void hitByTurtleShell() {          //public method for interaction
       nbLife--;
       checkDead();
   }
}
```

Accès au niveau de la classe

- C'est une erreur commune de croire que les modificateurs d'accès agissent sur les objets (e.g., sur les instances de classe)
- Parce que tous les objets d'une classe connaissent la structure de la classe, **un objet d'une certaine classe a accès aux attributs ou aux méthodes `private` d'un autre objet de la même classe**

```
public class NombreEntier {
     private int valeur;
     public int addition(NombreEntier nEnt) {
          int somme = valeur + nEnt.valeur;
          return somme;
     }
}
```

Attributs de classe : référent `this` (1/2)

- Dans le classe `Mario`, on a modifié la valeur d'un attribut via des méthodes
  - Comment sait-on sur quel objet le code agit?

    Le compilateur relie secrètement les attributs aux référents des objets
- Le référent spécial `this` permet de faire référence à **l'objet lui-même** à l'intérieur de sa propre classe

```
class TrafficLight
{
        int color;
        void changeColor (int color) {
                this.color = (this.color + 1);
                if (this.color > 3) this.color = 1;
        }
}
```

- Les attributs d'un objet sont directement accessibles dans ses méthodes : inutile de les passer explicitement en argument

Attributs de classe : référent `this` (2/2)

- **`this`** (l'objet lui-même) permet de **distinguer** les **attributs** des **arguments** de même nom, notamment dans l'utilisation du constructeur

```
class TrafficLight {
        int color;
        // Constructor (! no return type !)
        TrafficLight(int color) {
                this.color = color;
        }
        TrafficLight getInstance() {
                return this;
        }
}
```

- **`this`** est utilisé pour retourner un référent à l'objet courant

Encapsulation (3/3)

- Reprise de la classe `Mario` :

```
public class Mario {          // The class must be public to be used
	private int nbLife;   // No one but Mario objects can access this
	public Mario() {      // Constructor must be public to be called
		this.nbLife = 3;
	}
	private void checkDead() {          // internal method
		if(this.nbLife <= 0) {
			System.out.println("Game Over !");
		}
	}
	public void hitByTurtleShell() {    // public method for interaction
		this.nbLife--;
		checkDead();
	}
}
```

Avantage 1 : Préserver l'intégrité des objets

- En déclarant tous les attributs `private`, on confie à la classe seule **la responsabilité** de leur manipulation par le biais de méthodes (messages) particuliers :
  les accesseurs
  - les accesseurs en lecture pour récupérer la valeur de l'attribut (ex., `getSpeed`)
  - les accesseurs en écriture pour modifier la valeur de l'attribut (ex., `setSpeed`)

- Cela permet de **protéger les attributs** (surtout en écriture)

- Cela permet aussi de **gérer les erreurs d'utilisation**

Protéger les attributs

```
public class Homework {
	private int grade;
	public void setGrade(int grade) {
		if(grade >= 0 && grade <= 20)
			this.grade = grade;
	}
	public int getGrade() { return this.grade; }
}
```

```
Homework myHomework = new Homework();   //default constructor
myHomework.grade = 10;                  //KO
myHomework.setGrade(10);                //OK
```

Avantage 2 : Cloisonner l'évolution des classes

- L'encapsulation **limite l'interdépendance des classes**
  - méthodes `private` : fonctionnement interne, le prototype peut être **modifié sans impacter** les classes associées
  - méthodes `public`: **interface**, donc le prototype est non modifiable pour maintenir l'interopérabilité
    - très important lors de développements collaboratifs

Pensez à choisir le bon type d'arguments en prévoyant l'évolution future pour limiter le nombre de méthodes de l'interface (`public`)

Avantage 3 : Développement collaboratif

- Le découpage en modules faiblement interdépendants permet une **meilleure stabilité** générale du code, ainsi que la répartition de travail à plusieurs

- En revanche, cela implique d'**anticiper** la conception du programme pour se mettre d'accord a priori sur les différentes interfaces des classes

Exercice



Cours 2 - Question 7 et Question 8

Plan

1. Rappels
2. Abstraction
3. Allocation en mémoire
4. Héritage
5. Encapsulation
6. Cacher l'implémentation

Cacher l'implémentation

- En POO, on sépare les choses qui ne changent pas des choses qui changent
- Cela permet à un fournisseur de faire des modifications dans sa classe sans modifier les services que la classe offre à ses clients

- En Java, les **modificateurs d'accès** spécifient la visibilité d'une classe, d'un attribut ou d'une méthode

  `Public`

  `Private`

  `Protected`

  `Friendly` (par défaut) : sans mot-clef

- Comment définit-on cette visibilité ?

Les packages (1/3)

- Les classes sont regroupées en **packages**, qui constituent des unités de bibliothèque (*library units*)

Hierarchy For Package java.lang

Package Hierarchies:
All Packages

Class Hierarchy

- java.lang.**Object**
  - java.lang.**Boolean** (implements java.lang.Comparable<T>, java.io.Serializable)
  - java.lang.**Character** (implements java.lang.Comparable<T>, java.io.Serializable)
  - java.lang.**Character.Subset**
    - java.lang.**Character.UnicodeBlock**
  - java.lang.**Enum**<E> (implements java.lang.Comparable<T>, java.io.Serializable)
  - java.lang.**Math**
  - java.lang.**Number** (implements java.io.Serializable)
    - java.lang.**Byte** (implements java.lang.Comparable<T>)
    - java.lang.**Double** (implements java.lang.Comparable<T>)
    - java.lang.**Float** (implements java.lang.Comparable<T>)
    - java.lang.**Integer** (implements java.lang.Comparable<T>)
    - java.lang.**Long** (implements java.lang.Comparable<T>)
    - java.lang.**Short** (implements java.lang.Comparable<T>)

Les packages (2/3)

- Comment regrouper les classes dans des packages ?
  - Par le mot cléf **package** en début de fichier
  - Exemple :

```
package com.mycompany.myframework.complex
```

- Comment utiliser une classe dans un autre package ?
  - Par le mot cléf **import** en début de fichier
  - Exemple :

```
import com.mycompany.myframework.complex.*
```

Le symbole * signifie qu'on importe toutes les classes du package `complex`

Les packages (3/3)

- Les noms des packages doivent être uniques
  - cela enlève toute ambiguïté dans l'utilisation des classes en créant les *namespace*

- Comment l'interpréteur Java trouve-t-il les classes ?
  - Il cherche la variable d'environnement CLASSPATH qui donne le répertoire qui contient la racine d'une recherche des fichiers `.class`
  - La recherche se fait en prenant les noms des packages et en remplaçant les séparateurs `.` par des séparateurs `\`
  - Exemple :

```
CLASSPATH = D:\JAVA\LIB
package com.esiea.javaframework.complex

        =>  D:\JAVA\LIB\com\esiea\javaframework\complex
```

Les packages et les modificateurs d'accès

Modificateur d'accès	Visibilité des éléments
`Public`	Classes à l'intérieur et à l'extérieur du package
`(friendly)`	Classes à l'intérieur du package
`Protected`	Classe courante et ses classes filles, à l'intérieur et à l'extérieur du package
`Private`	Classe courante


esiea

Les avantages de la POO

Fonctionnalités	Conséquences
la représentation objet	. réduit la complexité du programme . la structure du programme est claire
chaque objet est une entité à part	. la modification, la localisation et la correction des programmes sont faciles
les changements dans une classe n'affectent pas les autres	. facilité d'écriture et de maintenance
l'intégrité et la sécurité des données sont renforcées par certains langages	. programmes sécurisés, fiables, avec moins de risques d'erreurs
les nouvelles fonctionnalités sont introduites grâce à l'héritage et sans modifier l'existant	. les programmes évoluent facilement
le code est facilement réutilisé	. temps de développement réduit . qualité des programmes améliorée
les programmes sont plus faciles à adapter et à mettre à l'échelle	. des systèmes à grande échelle peuvent être crées en assemblant des sous-systèmes réutilisables